SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO3: *Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution, chaining methods, knowledge representation to perform logical reasoning for drawing valid conclusions and decision making. (BL4)*

Assessment Tool Weightage: 50%

Time Duration: 1 Hour

QUESTIONS: 4 **Total Marks:40**

Annexure A

CASE STUDY

Code of Conduct:

I DEEPESH (192511464)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
DEEPESH

ANNEXURE A
CASE STUDY 1 — SMART MEDICAL DIAGNOSIS SYSTEM
Case Study

A hospital is designing a rule-based Logical Agent for preliminary patient diagnosis. The system's knowledge base contains the following rules and facts:

1. If a patient has Fever AND Cough → Possible Flu.
2. If a patient has Fever AND Rash → Possible Measles.
3. If a patient has Cough AND Breathlessness → Possible Pneumonia.
4. Facts about Patient A:
   - Fever = True
   - Cough = True
   - Breathlessness = True

(a) Represent the Above Rules and Facts Precisely Using Propositional Logic Symbols. Define Each Symbol Used.
Propositional Logic Representation

Propositional Logic represents statements using symbols that can have either a True or False value.

Let the following symbols represent the patient's conditions and diagnoses:

F = Patient A has Fever

C = Patient A has Cough

B = Patient A has Breathlessness

R = Patient A has Rash

FL = Patient A has Possible Flu

M = Patient A has Possible Measles

P = Patient A has Possible Pneumonia

Meaning of Logical Symbols

∧ = AND

→ = IMPLIES / IF-THEN

¬ = NOT

= True or False value of a proposition

Representation of Rule 1

Statement:

If a patient has Fever AND Cough, then Possible Flu.

Logical representation:

$F \land C \rightarrow FL$

This means that if both Fever and Cough are true, then Possible Flu is concluded.

Representation of Rule 2

Statement:

If a patient has Fever AND Rash, then Possible Measles.

Logical representation:

$F \land R \rightarrow M$

This means that if both Fever and Rash are true, then Possible Measles is concluded.

Representation of Rule 3

Statement:

If a patient has Cough AND Breathlessness, then Possible Pneumonia.

Logical representation:

$C \land B \rightarrow P$

This means that if both Cough and Breathlessness are true, then Possible Pneumonia is concluded.

Representation of Facts

The given facts about Patient A are:

Fever = True

Therefore:

F = True

Cough = True

Therefore:

C = True

Breathlessness = True

Therefore:

B = True

There is no given fact that Patient A has Rash.

Therefore:

R = Unknown

Complete Knowledge Base

The complete propositional knowledge base can be written as:

$F \land C \rightarrow FL$

$F \land R \rightarrow M$

$C \land B \rightarrow P$

F

C

B

Therefore, the knowledge base contains three rules and three initial facts.

(b) Apply Forward Chaining on the Knowledge Base to Derive All Possible Diagnoses for Patient A

Definition of Forward Chaining

Forward Chaining is a data-driven reasoning method.

It starts with the known facts and repeatedly applies rules whose conditions are satisfied. Whenever a rule's conditions are true, its conclusion is added as a new fact.

For Patient A, the initial facts are:

F = True

C = True

B = True

Step 1: Start with Initial Facts

The initial knowledge about Patient A is:

Fever(A)

Cough(A)

Breathlessness(A)

Therefore:

F = True

C = True

B = True

Step 2: Check Rule 1

Rule 1 is:

$F \wedge C \rightarrow FL$

The conditions required are:

F = True

C = True

We know that:

F = True

and:

C = True

Therefore, both conditions are satisfied.

Hence Rule 1 can be fired.

Inference

$F \wedge C$

$\downarrow$

FL

Therefore:

FL = True

Step 3: Check Rule 2

Rule 2 is:

$F \land R \rightarrow M$

The conditions required are:

$F = True$

$R = True$

We know:

$F = True$

However, there is no fact stating:

$R = True$

That is, Rash is not given for Patient A.

Therefore, Rule 2 cannot be fired.

Hence:

Possible Measles cannot be derived.

Step 4: Check Rule 3

Rule 3 is:

$C \land B \rightarrow P$

The conditions required are:

$C = True$

$B = True$

We know:

$C = True$

and:

$B = True$

Therefore, both conditions are satisfied.

Hence Rule 3 can be fired.

Inference

$C \land B$

↓

P

Therefore:

P = True

So:

Possible Pneumonia is derived.

Forward Chaining Inference Sequence

The complete inference process can be represented as:

```
Initial Facts
     |
     +------------------+
     |                  |
     v                  v
Fever(A)            Cough(A)
     |                  |
     +--------+---------+
              |
              v
       Rule 1: F ∧ C → FL
              |
              v
       Possible Flu
```

For Pneumonia:

```
Cough(A) + Breathlessness(A)
              |
              v
       Rule 3: C ∧ B → P
              |
              v
       Possible Pneumonia
```

For Measles:

```
Fever(A) + Rash(A)
        |
        v
Rule 2: F ∧ R → M

Rash(A) is not given
        |
        v
Rule cannot be fired
        |
        v
Possible Measles NOT derived’
```

Forward Chaining Table

Step	Rule Checked	Conditions	Result
1	Initial facts	F, C, B are True	Start
2	Rule 1	F ∧ C = True	Possible Flu
3	Rule 2	F ∧ R; R is unknown	Not fired
4	Rule 3	C ∧ B = True	Possible Pneumonia

Final Diagnoses from Forward Chaining

The system successfully derives:

1. Possible Flu
2. Possible Pneumonia

The system cannot derive:

3. Possible Measles

because Rash is not available as a fact.

Therefore:

Final Diagnosis: Patient A has Possible Flu and Possible Pneumonia.

(c) Apply Backward Chaining to Confirm "Patient A Has Pneumonia"

Definition of Backward Chaining

Backward Chaining is a goal-driven reasoning technique.

It starts with the desired goal and works backward to determine which rules can produce that goal. It then checks whether the conditions of those rules are known to be true.

Step 1: Define the Goal

The goal is:

Pneumonia(A)

Using the propositional symbol:

P

Therefore, our goal is:

P = True

Step 2: Find a Rule That Produces Pneumonia

Look at the given rules.

Rule 1:

$F \land C \to FL$

This produces Flu, not Pneumonia.

Rule 2:

$F \land R \to M$

This produces Measles, not Pneumonia.

Rule 3:

$C \land B \to P$

This produces Pneumonia.

Therefore, Rule 3 is selected.

Step 3: Reduce the Goal

Rule 3 is:

$C \land B \to P$

To prove:

P

we need to prove:

$C \land B$

Therefore, the original goal:

Pneumonia(A)

is reduced into two subgoals:

Cough(A)

and:

Breathlessness(A)

Step 4: Check the First Subgoal

First subgoal:

Cough(A)

The knowledge base contains:

Cough(A) = True

Therefore, the first subgoal is satisfied.

Hence:

C = True

Step 5: Check the Second Subgoal

Second subgoal:

Breathlessness(A)

The knowledge base contains:

Breathlessness(A) = True

Therefore, the second subgoal is satisfied.

Hence:

B = True

Step 6: Satisfy the Main Goal

We have proved:

Cough(A) = True

and:

Breathlessness(A) = True

Therefore:

$C \land B = \text{True}$

Using Rule 3:

$C \land B \rightarrow P$

we conclude:

P = True

Backward Chaining Goal Reduction Tree

```
                  GOAL
             Pneumonia(A)
                  |
                  |
       Rule 3: C ∧ B → P
                  |
                  v
           Cough(A) AND
        Breathlessness(A)
               /           \
              /             \
             v               v
     Cough(A)       Breathlessness(A)
        |                    |
      TRUE                 TRUE
              \             /
               \           /
                \         /
                 \       /
            Pneumonia(A)
                TRUE
```

Detailed Backward Chaining Steps

Goal 1

Pneumonia(A)

Search for a rule whose conclusion is Pneumonia(A).

Found:

Cough(A) ∧ Breathlessness(A) → Pneumonia(A)

Therefore, create two subgoals:

Cough(A)

Breathlessness(A)

]

Subgoal 1

Cough(A)

Given directly as a fact.

Therefore:

Cough(A) = True

Subgoal 1 is satisfied.

Subgoal 2

Breathlessness(A)

Given directly as a fact.

Therefore:

Breathlessness(A) = True

Subgoal 2 is satisfied.

Final Goal

Since both subgoals are true:

Cough(A) ∧ Breathlessness(A)

Therefore:

Pneumonia(A)

Hence:

Patient A has Pneumonia.

]

Comparison of Forward and Backward Chaining

Forward Chaining

Forward Chaining starts from the available facts:

Fever + Cough + Breathlessness

Then it applies the rules.

It derives:

Possible Flu

Possible Pneumonia

Backward Chaining

Backward Chaining starts from the goal:

Pneumonia(A)

It searches for a rule that produces Pneumonia.

It finds:

Cough ∧ Breathlessness → Pneumonia

It then checks whether:

Cough(A)

and:

Breathlessness(A)

are true.

Both are given facts.

Therefore, the goal is proved.

CASE STUDY 2 — AUTONOMOUS TRAFFIC MANAGEMENT AGENT

A smart city traffic controller uses a First-Order Logic (FOL) based knowledge base to manage emergency vehicle routing.

Given Rules

Rule 1:

$\forall x : Vehicle(x) \land EmergencyType(x) \rightarrow ClearPath(x)$

Rule 2:

$\forall x : ClearPath(x) \rightarrow GreenSignal(x) \land \neg RedSignal(x)$

Rule 3:

$\forall x \forall y : Vehicle(x) \land Behind(x,y) \land GreenSignal(y) \rightarrow Proceed(x)$

Given Facts

Vehicle(Ambulance)

EmergencyType(Ambulance)

Vehicle(CarA)

Behind(CarA, Ambulance)

(a) Unification

We first consider Rule 1:

$Vehicle(x) \land EmergencyType(x) \rightarrow ClearPath(x)$

The first condition is:

Vehicle(x)

The corresponding fact is:

Vehicle(Ambulance)

Therefore, x is substituted by Ambulance.

The substitution is:

$\theta = \{x/Ambulance\}$

Now check the second condition:

EmergencyType(x)

After applying the substitution:

EmergencyType(Ambulance)

This matches the given fact:

EmergencyType(Ambulance)

Therefore, the complete substitution is:

$\theta = \{x/\text{Ambulance}\}$

Applying this substitution to the conclusion:

ClearPath(x)

gives:

ClearPath(Ambulance)

Unification Steps

Step 1:

Vehicle(x) matches Vehicle(Ambulance)

Therefore:

x/Ambulance

Step 2:

EmergencyType(x) becomes EmergencyType(Ambulance)

This matches the fact.

Step 3:

ClearPath(x) becomes:

ClearPath(Ambulance)

Most General Unifier

MGU:

$\theta = \{x/\text{Ambulance}\}$

Therefore:

ClearPath(Ambulance)

is derived.

(b) Resolution Proof of Proceed(CarA)

The goal is:

Proceed(CarA)

To prove this using Resolution, convert the relevant rules into CNF.

]

Step 1: Convert Rule 1 to CNF

Original rule:

Vehicle(x) ∧ EmergencyType(x) → ClearPath(x)

Using:

A → B ≡ ¬A ∨ B

we obtain:

¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)

Therefore:

C1 = ¬Vehicle(x) ∨ ¬EmergencyType(x) ∨ ClearPath(x)

]

Step 2: Convert Rule 2 to CNF

Original:

ClearPath(x) → GreenSignal(x) ∧ ¬RedSignal(x)

This gives two clauses:

C2 = ¬ClearPath(x) ∨ GreenSignal(x)

C3 = ¬ClearPath(x) ∨ ¬RedSignal(x)

Step 3: Convert Rule 3 to CNF

Original:

Vehicle(x) ∧ Behind(x,y) ∧ GreenSignal(y) → Proceed(x)

Therefore:

C4 = ¬Vehicle(x) ∨ ¬Behind(x,y) ∨ ¬GreenSignal(y) ∨ Proceed(x)

Step 4: Use the Given Facts

F1 = Vehicle(Ambulance)

F2 = EmergencyType(Ambulance)

F3 = Vehicle(CarA)

F4 = Behind(CarA, Ambulance)

Step 5: Derive ClearPath(Ambulance)

From C1:

¬Vehicle(Ambulance) ∨ ¬EmergencyType(Ambulance) ∨ ClearPath(Ambulance)

Using:

Vehicle(Ambulance)

and:

EmergencyType(Ambulance)

by Resolution:

ClearPath(Ambulance)

Step 6: Derive GreenSignal(Ambulance)

From C2:

¬ClearPath(Ambulance) ∨ GreenSignal(Ambulance)

We already derived:

ClearPath(Ambulance)

Therefore:

GreenSignal(Ambulance)

Step 7: Derive Proceed(CarA)

From C4, substitute:

x = CarA

y = Ambulance

We get:

¬Vehicle(CarA) ∨ ¬Behind(CarA,Ambulance) ∨ ¬GreenSignal(Ambulance) ∨ Proceed(CarA)

We have:

Vehicle(CarA)

Behind(CarA,Ambulance)

GreenSignal(Ambulance)

Therefore, by Resolution:

Proceed(CarA)

Resolution Chain

```
                    Vehicle(Ambulance)
                           +
                 EmergencyType(Ambulance)
                           |
                           v
                  ClearPath(Ambulance)
                           |
                           v
                 GreenSignal(Ambulance)
                           |
             +---------------------------------+
             |                                 |
 Vehicle(CarA)            Behind(CarA,Ambulance)
             |                                 |
             +----------------+----------------+
                              |
                              v
                        Proceed(CarA)
```

Conclusion

Therefore:

PROCEED(CarA) IS PROVED.

(c) Handling Two Simultaneous Emergency Vehicles

The current FOL knowledge base is not completely sufficient to handle two simultaneous emergency vehicles.

Suppose there are two emergency vehicles:

Vehicle(Ambulance1)

EmergencyType(Ambulance1)

Vehicle(Ambulance2)

EmergencyType(Ambulance2)

Using Rule 1, the system can derive:

ClearPath(Ambulance1)

and:

ClearPath(Ambulance2)

Using Rule 2, the system can derive:

GreenSignal(Ambulance1)

and:

GreenSignal(Ambulance2)

However, the current knowledge base does not specify:

1. Which emergency vehicle gets priority.
2. How conflicting routes are handled.
3. How two emergency vehicles are separated.
4. How normal traffic should be stopped.
5. What happens if both vehicles approach the same intersection.
6. Which vehicle should proceed first.

Additional Rules Required

A priority rule could be added:

$$EmergencyType(x) \land EmergencyType(y) \land Conflict(x,y) \rightarrow Priority(x)$$

A traffic stopping rule could be:

$$Priority(x) \land Conflict(x,y) \rightarrow Stop(y)$$

A separation rule could be:

$$EmergencyType(x) \land EmergencyType(y) \land SameIntersection(x,y) \rightarrow MaintainDistance(x,y)$$

Conclusion

Therefore, additional priority, conflict detection, and traffic-control rules are required to safely handle two simultaneous emergency vehicles.

CASE STUDY 3 — AGRICULTURAL EXPERT REASONING SYSTEM

An agricultural advisory logical agent operates with the following propositional knowledge base.

Rules

P1:

SoilDry → IrrigationNeeded

P2:

IrrigationNeeded ∧ CropWheat → ApplyDripMethod

P3:

¬ApplyDripMethod ∧ CropWheat → CropAtRisk

Facts

SoilDry = True

CropWheat = True

(a) Conversion into CNF

CNF means Conjunctive Normal Form.

A formula is in CNF when it is represented as an AND of OR clauses.

P1 Conversion

Original:

SoilDry → IrrigationNeeded

Using:

A → B ≡ ¬A ∨ B

Therefore:

¬SoilDry ∨ IrrigationNeeded

Hence:

C1 = ¬SoilDry ∨ IrrigationNeeded

P2 Conversion

Original:

IrrigationNeeded ∧ CropWheat → ApplyDripMethod

Using:

A ∧ B → C

becomes:

¬A ∨ ¬B ∨ C

Therefore:

¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

Hence:

C2 = ¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

P3 Conversion

Original:

¬ApplyDripMethod ∧ CropWheat → CropAtRisk

First eliminate the implication:

¬(¬ApplyDripMethod ∧ CropWheat) ∨ CropAtRisk

Using De Morgan's Law:

ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

Therefore:

C3 = ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

Facts in CNF

Fact 1:

C4 = SoilDry

Fact 2:

C5 = CropWheat

Final CNF Representation

C1:

¬SoilDry ∨ IrrigationNeeded

C2:

¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

C3:

ApplyDripMethod ∨ ¬CropWheat ∨ CropAtRisk

C4:

SoilDry

C5:

CropWheat

(b) Resolution Refutation to Prove ApplyDripMethod

Goal

ApplyDripMethod

For Resolution Refutation, negate the goal.

Therefore:

¬ApplyDripMethod

is added to the knowledge base.

Step 1

We have:

C1:

¬SoilDry ∨ IrrigationNeeded

and:

C4:

SoilDry

Resolving these clauses gives:

IrrigationNeeded

Therefore:

IrrigationNeeded is derived.

Step 2

We have:

C2:

¬IrrigationNeeded ∨ ¬CropWheat ∨ ApplyDripMethod

We also have:

IrrigationNeeded

and:

CropWheat

Therefore, resolving the clauses gives:

ApplyDripMethod

Step 3

The negated goal is:

¬ApplyDripMethod

We have derived:

ApplyDripMethod

Resolving:

ApplyDripMethod

with:

¬ApplyDripMethod

gives the empty clause:

□

The empty clause represents a contradiction.

Therefore, the negated goal is false and the original goal is true.

Resolution Proof Tree

```
                          SoilDry
                             |
                             |
           ¬SoilDry ∨ IrrigationNeeded
                             |
                             v
                   IrrigationNeeded
                             |
                             |
      ¬IrrigationNeeded ∨ ¬CropWheat
               ∨ ApplyDripMethod
                             |
                         CropWheat
                             |
                             v
                   ApplyDripMethod
                             |
                             |
                   ¬ApplyDripMethod
                     (Negated Goal)
                             |
                             v
```

(c) Removing the Fact SoilDry

Suppose the fact:

SoilDry = True

is removed from the knowledge base.

The remaining fact is:

CropWheat = True

We still have Rule P1:

SoilDry → IrrigationNeeded

or in CNF:

¬SoilDry ∨ IrrigationNeeded

However, SoilDry is no longer known to be true.

Therefore, we cannot derive:

IrrigationNeeded

Without IrrigationNeeded, Rule P2 cannot be applied:

IrrigationNeeded ∧ CropWheat → ApplyDripMethod

Therefore:

ApplyDripMethod cannot be derived.

CASE STUDY 4 — WUMPUS WORLD KNOWLEDGE AGENT

An AI agent is navigating the Wumpus World.

Given Percepts

1. Stench at [1,2]
2. Breeze at [1,1]
3. Glitter at [2,2]

Rules

Rule 1:

Stench[1,2] → Wumpus is adjacent to [1,2]

Rule 2:

Breeze[1,1] → Pit is adjacent to [1,1]

Rule 3:

Glitter[x,y] → Gold[x,y]

Rule 4:

¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]

(a) Construct the Belief State and Apply Modus Ponens

The agent initially perceives:

Stench[1,2]

Breeze[1,1]

Glitter[2,2]

These percepts form the initial belief state.

Step 1: Stench Percept

Rule:

Stench[1,2] → WumpusAdjacent[1,2]

Given:

Stench[1,2]

By Modus Ponens:

WumpusAdjacent[1,2]

Therefore:

Wumpus is adjacent to [1,2].

Step 2: Breeze Percept

Rule:

Breeze[1,1] → PitAdjacent[1,1]

Given:

Breeze[1,1]

By Modus Ponens:

PitAdjacent[1,1]

Therefore:

A Pit is adjacent to [1,1].

Step 3: Glitter Percept

Rule:

Glitter[x,y] → Gold[x,y]

Given:

Glitter[2,2]

Substituting:

$x = 2$

$y = 2$

Therefore:

Gold[2,2]

Hence:

Gold is located at [2,2].

Step 4: Safety Rule

The safety rule is:

¬Wumpus[x,y] ∧ ¬Pit[x,y] → Safe[x,y]

To apply this rule, the agent must know that a particular cell does not contain a Wumpus and does not contain a Pit.

Such negative facts are not given.

Therefore, the safety rule cannot be applied at this stage.

Derived Conclusions

1. WumpusAdjacent[1,2]
2. PitAdjacent[1,1]
3. Gold[2,2]

No cell can yet be conclusively declared safe.

(b) Forward Chaining to Identify Possible Wumpus and Pit Locations

We assume a standard Wumpus World grid in which cells are adjacent vertically and horizontally.

Wumpus Locations

We have:

Stench[1,2]

According to Rule 1:

Wumpus is adjacent to [1,2].

The neighboring cells of [1,2] are:

[1,1]

[1,3]

[2,2]

Therefore:

Wumpus ∈ {[1,1], [1,3], [2,2]}

These are possible Wumpus locations.

Reasoning
```
Stench[1,2]
       |
       v
Wumpus adjacent to [1,2]
       |
       +-----------------+-----------------+
       |                 |                 |
     [1,1]             [1,3]             [2,2]
   Possible          Possible          Possible
```

Pit Locations

We have:

Breeze[1,1]

According to Rule 2:

Pit is adjacent to [1,1].

The neighboring cells are:

[1,2]

[2,1]

Therefore:

Pit ∈ {[1,2], [2,1]}

These are possible Pit locations.

Reasoning
```
Breeze[1,1]
       |
       v
Pit adjacent to [1,1]
       |
       +-----------------+
       |                 |
     [1,2]             [2,1]
   Possible          Possible
```
\

Gold Location

We have:

Glitter[2,2]

Using Rule 3:

Glitter[x,y] → Gold[x,y]

Therefore:

Gold[2,2]

Hence:

Gold is definitely at [2,2].

(c) Evaluate the Safety of the Path

The proposed path is:

[1,1] → [1,2] → [2,2]

The goal is to determine whether this path is safe.

Cell [1,1]

The agent perceives:

Breeze[1,1]

This means a Pit may be in an adjacent cell.

It does not prove that [1,1] itself contains a Pit.

However, the available knowledge is not sufficient to prove:

Safe[1,1]

because the required negative facts are not available.

Cell [1,2]

The agent perceives:

Stench[1,2]

Therefore, a Wumpus is adjacent to [1,2].

Also, because there is a Breeze at [1,1], a Pit may be at:

[1,2]

or:

[2,1]

Therefore, [1,2] is a possible Pit location.

Hence:

[1,2] is not proven safe.

Cell [2,2]

The agent perceives:

Glitter[2,2]

Therefore:

Gold[2,2]

However, Glitter does not prove that the cell is free from a Wumpus or Pit.

The safety rule requires:

¬Wumpus[2,2]

and:

¬Pit[2,2]

These facts are not available.

Therefore, [2,2] cannot be conclusively proven safe.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand